

TECH

BUILD TOOLS / BUNDLING

BUILD TOOLS

BUILD TOOLS

30 ВОПРОСОВ 7 ДНЕЙ КОНСПЕКТ ЛЕКЦИЙ

ИСТОЧНИК vitejs.dev / webpack.js.org / [mdn](https://mdn.io)

20 ВОПРОСОВ 7 ДНЕЙ

ДЕНЬ 1	BUNDLER	VITE vs WEBPACK	DEV SERV	BUNDLER	VITE vs WEBPACK	DEV SERV
СРЕДНИЙ	ER ESM	HMR		ER ESM	HMR	
	Q1 · Q2 · Q3 · Q4 · Q5 · Q6					

ДЕНЬ 2	VITE PROD BUILD	ROLLUP	ESBUILD	VITE PROD BUILD	ROLLUP	ESBUILD
СРЕДНИЙ	Q7 · Q8 · Q9					

ДЕНЬ 3	TREE SHAKING	SIDE EFFECTS	CODE S	TREE SHAKING	SIDE EFFECTS	CODE S
СРЕДНИЙ	PLITTING			PLITTING		
	Q10 · Q11 · Q12					

ДЕНЬ 4	DYNAMIC IMPORT	CHUNK	CHUNK SPLIT	DYNAMIC IMPORT	CHUNK	CHUNK SPLIT
СРЕДНИЙ	TING BUNDLE SIZE			TING BUNDLE SIZE		
	Q13 · Q14 · Q15 · Q16 · Q17					

ДЕНЬ 5	SOURCE MAPS	CACHE BUSTING	LONG-T	SOURCE MAPS	CACHE BUSTING	LONG-T
СРЕДНИЙ	ERM CACHING			ERM CACHING		
	Q18 · Q19 · Q20 · Q21					

ДЕНЬ 6	DEPS	POLYFILLS	LEGACY	ENV	DEPS	POLYFILLS	LEGACY	ENV
СРЕДНИЙ	Q22 · Q23 · Q24 · Q25 · Q26							

ДЕНЬ 7	BUILD ENVS	WHITE-LABEL	SHARED PK	BUILD ENVS	WHITE-LABEL	SHARED PK
ТЯЖЁЛЫЙ	G DEDUP			G DEDUP		
	Q27 · Q28 · Q29 · Q30					

РИТМ 30-60 МИН ТЕОРИЯ 15-30 МИН КОД В КОНСОЛИ 15 МИН ОТВЕТЫ ВСЛУХ

ДЕНЬ 7 — БЕЗ НОВОЙ ТЕОРИИ ТОЛЬКО ПРОГОН ПО 20 ВОПРОСАМ

ДЕНЬ 1

НАГРУЗКА

СРЕДНИЙ

6 ВОПРОСОВ

BUNDLER**VITE vs WEBPACK****DEV SERVER****ESM****HMR**

ЧТО УЧИМ В ЭТОТ ДЕНЬ

BUNDLER

VITE vs WEBPACK

DEV SERVER

ESM

HMR

ВОПРОСЫ ДНЯ

Q01 Что делает bundler?

Q02 Чем Vite отличается от Webpack?

Q03 Как работает Vite dev server?

Q04 Что такое native ESM?

Q05 Что такое HMR?

Q06 Почему HMR может ломаться?

Что делает bundler?

ОПИСАНИЕ

Объединяет модули JS/CSS/assets в bundle для деплоя. Решает: разрешение модулей, transpile (TS/Babel), tree-shake, minify, code-splitting, asset optimization, source-maps.

КАК РАБОТАЕТ

- 01 Resolve modules.
- 02 Transpile (TS, Babel).
- 03 Tree-shake unused.
- 04 Minify + chunks.
- 05 Asset processing.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Любое production frontend.
- 02 SPA, library.

ПОДВОДНЫЕ КАМНИ

- 01 Без bundler – N HTTP requests.
- 02 Без tree-shake – лишний код в bundle.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое module resolution?
- 02 Чем Vite от Webpack отличается?

ПОЛНАЯ СТАТЬЯ

<https://vitejs.dev/guide/why.html>

ВОПРОС Q02

Чем Vite отличается от Webpack?

ОПИСАНИЕ

Vite использует native ESM в dev (мгновенный старт, on-demand transform). Webpack бандлит весь dev-bundle. Production: Vite использует Rollup, Webpack – свой bundler. Vite быстрее в dev, primarily через esbuild.

КАК РАБОТАЕТ

- 01 Vite: native ESM dev.
- 02 Webpack: pre-bundled dev.
- 03 Vite prod: Rollup.
- 04 Webpack prod: own bundler.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Новые проекты → Vite.
- 02 Existing complex Webpack → migrate cautiously.

ПОДВОДНЫЕ КАМНИ

- 01 Vite в legacy браузерах сложнее.
- 02 Webpack-plugins не работают в Vite.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое native ESM?
- 02 Чем Rollup от Webpack отличается?

ПОЛНАЯ СТАТЬЯ

<https://vitejs.dev/guide/why.html>

Как работает Vite dev server?

ОПИСАНИЕ

Запускается на native ESM в браузере. Каждый `import` – отдельный HTTP-запрос. Vite перехватывает, transpile on-demand (esbuild), кэширует. HMR через ESM.

КАК РАБОТАЕТ

- 01 Native ESM in dev.
- 02 On-demand transpile (esbuild).
- 03 Per-module HMR.
- 04 Cold start ~ms.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Vite-проекты в dev.
- 02 Migration с Webpack.

ПОДВОДНЫЕ КАМНИ

- 01 Network tab показывает много модулей.
- 02 Старые npm-пакеты в CommonJS – pre-bundle.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое pre-bundling?
- 02 Чем esbuild отличается?

ПОЛНАЯ СТАТЬЯ

<https://vitejs.dev/guide/dep-pre-bundling.html>

ВОПРОС Q04

Что такое native ESM?

ОПИСАНИЕ

JavaScript Modules – стандартный формат с `import/export`, поддерживаемый браузерами. Vite использует это в dev, без bundling. `<script type="module" src="app.js">`.

КАК РАБОТАЕТ

- 01 `import/export` syntax.
- 02 `<script type="module">`.
- 03 Browser-supported.
- 04 Strict mode by default.
- 05 Top-level await.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Modern browsers.
- 02 Vite dev.
- 03 Edge functions.

ПОДВОДНЫЕ КАМНИ

- 01 IE / old Safari – нет.
- 02 CORS для cross-origin imports.

МОГУТ ДОСПРОСИТЬ

- 01 Чем CommonJS от ESM отличается?
- 02 Что такое import maps?

ПОЛНАЯ СТАТЬЯ

<https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Modules>

Что такое HMR?

ОПИСАНИЕ

Hot Module Replacement – замена модуля в работающем приложении без full reload. Сохраняет state. Реализация – bundler watches файлы, при изменении – отправляет patch в браузер.

КАК РАБОТАЕТ

- 01 File watcher → bundler → browser.
- 02 Replace module without reload.
- 03 Preserve state.
- 04 React Fast Refresh – React-aware HMR.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Dev workflow.
- 02 React / Vue / Svelte.

ПОДВОДНЫЕ КАМНИ

- 01 Module без HMR-API – full reload.
- 02 HMR с side-effects может ломаться.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое React Fast Refresh?
- 02 Когда HMR ломается?

ПОЛНАЯ СТАТЬЯ

<https://vitejs.dev/guide/api-hmr.html>

ВОПРОС Q06

Почему HMR может ломаться?

ОПИСАНИЕ

Side-effects на module-level, store не HMR-aware, cyclic dependencies, API-changes (rename export), error в module. Реакция – full reload. Dev должен видеть warning.

КАК РАБОТАЕТ

- 01 Module-level side-effects.
- 02 Non-HMR-aware store.
- 03 Cyclic deps.
- 04 Rename export.
- 05 Module error.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Дебаг HMR-проблем.

ПОДВОДНЫЕ КАМНИ

- 01 Без warning разработчик думает что всё ок.
- 02 Stale state в HMR – confusing.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое HMR boundary?
- 02 Чем accept от dispose отличается?

ПОЛНАЯ СТАТЬЯ

<https://vitejs.dev/guide/api-hmr.html>

VITE PROD BUILD

ROLLUP

ESBUILD

ЧТО УЧИМ В ЭТОТ ДЕНЬ

VITE PROD BUILD

ROLLUP

ESBUILD

ВОПРОСЫ ДНЯ

Q07 Как работает production build в Vite?

Q08 Что такое Rollup?

Q09 Что такое esbuild?

ВОПРОС Q 07

Как работает production build в Vite?

ОПИСАНИЕ

Vite использует Rollup для production. Делает: tree-shake, minify (esbuild / terser), code-splitting, asset hashing, generate manifest. Output – папка dist готовая к deploy.

КАК РАБОТАЕТ

- 01 Rollup as production bundler.
- 02 esbuild / terser minify.
- 03 Code-splitting + chunks.
- 04 Asset hashing for cache-busting.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Любой Vite-проект перед deploy.

ПОДВОДНЫЕ КАМНИ

- 01 Dev и prod ведут себя по-разному (CSS handling, CommonJS).
- 02 Sourcemaps в prod = публичный код.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое manifest.json?
- 02 Чем terser от esbuild отличается?

ПОЛНАЯ СТАТЬЯ

<https://vitejs.dev/guide/build.html>

Что такое Rollup?

ОПИСАНИЕ

Bundler для libraries. Фокус на tree-shake и небольших output. Использует ESM для лучшего tree-shake. Vite использует его в prod-build. Rollup-plugins – экосистема.

КАК РАБОТАЕТ

- 01 ES modules первый класс.
- 02 Tree-shake aggressive.
- 03 For libraries (npm packages).
- 04 Rollup-plugins.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 npm packages.
- 02 Vite prod-builds.
- 03 Components-lib.

ПОДВОДНЫЕ КАМНИ

- 01 Сложные SPA – Webpack часто проще.
- 02 Plugin compatibility между Vite/Rollup.

МОГУТ ДОСПРОСИТЬ

- 01 Чем Rollup от Webpack отличается?
- 02 Что такое UMD / ESM output?

ПОЛНАЯ СТАТЬЯ

<https://rollupjs.org/>

ВОПРОС 009

Что такое esbuild?

ОПИСАНИЕ

Сверхбыстрый bundler/minifier на Go. Используется для transpile в Vite (dev), для minify в production. Десятки раз быстрее JS-bundlers. Не такой гибкий как Webpack.

КАК РАБОТАЕТ

- 01 Written in Go.
- 02 10-100x faster than JS bundlers.
- 03 Transpile + minify.
- 04 Used by Vite (dev), tsup, Bun.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 TS-transpile.
- 02 Minification.
- 03 Library bundling.

ПОДВОДНЫЕ КАМНИ

- 01 Меньше плагинов чем Webpack.
- 02 Не делает full type-check (только transpile).

МОГУТ ДОСПРОСИТЬ

- 01 Чем esbuild от swc отличается?
- 02 Где транспиляция типы потеряет?

ПОЛНАЯ СТАТЬЯ

<https://esbuild.github.io/>

TREE SHAKING

SIDE EFFECTS

CODE SPLITTING

ЧТО УЧИМ В ЭТОТ ДЕНЬ

TREE SHAKING

SIDE EFFECTS

CODE SPLITTING

ВОПРОСЫ ДНЯ

Q10 Что такое tree shaking?

Q11 Почему tree shaking может не сработать?

Q12 Что такое side effects в package.json?

ВОПРОС Q10

Что такое tree shaking?

ОПИСАНИЕ

Удаление мёртвого кода (unused exports) из bundle. Работает с ESM (статический анализ). Bundler анализирует import/export граф и убирает то, что нигде не used.

КАК РАБОТАЕТ

- 01 Static analysis ESM imports.
- 02 Remove unused exports.
- 03 Marketed as 'dead code elimination'.
- 04 Requires ESM, not CommonJS.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Production builds.
- 02 npm packages с многими exports.

ПОДВОДНЫЕ КАМНИ

- 01 CommonJS не tree-shake-able.
- 02 Side-effects ломают tree-shake.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое side effects: false?
- 02 Чем CommonJS блокирует tree-shake?

ПОЛНАЯ СТАТЬЯ

<https://webpack.js.org/guides/tree-shaking/>

Почему tree shaking может не сработать?

ОПИСАНИЕ

Side-effects в impotrax (running code on import). CommonJS вместо ESM. Reexports без types. Dynamic imports с нестатическими путями. Misconfigured side-effects: false.

КАК РАБОТАЕТ

- 01 Module side-effects.
- 02 CommonJS imports.
- 03 Dynamic import paths.
- 04 Wrong sideEffects flag.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Bundle audit.
- 02 Library publishing.

ПОДВОДНЫЕ КАМНИ

- 01 sideEffects: false когда есть side-effects → broken bundle.
- 02 Polyfills с side-effects режутся.

МОГУТ ДОСПРОСИТЬ

- 01 Что писать в sideEffects?
- 02 Как проверить tree-shake?

ПОЛНАЯ СТАТЬЯ

<https://webpack.js.org/guides/tree-shaking/#mark-the-file-as-side-effect-free>

ВОПРОС Q12

Что такое side effects в package.json?

ОПИСАНИЕ

Поле "sideEffects": false в package.json говорит bundler-у, что модули не имеют side-effects при import → tree-shake безопасен. Можно массив для исключений: ["*.css"].

КАК РАБОТАЕТ

- 01 sideEffects: false → safe to tree-shake.
- 02 Array для explicit: ["*.css", "polyfill.js"].
- 03 Lib publishers – обязательно.
- 04 Default: true (чтобы не сломать).

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Library development.
- 02 Modern monorepos.

ПОДВОДНЫЕ КАМНИ

- 01 false когда есть side-effects → broken.
- 02 CSS-imports без declaration → теряются.

МОГУТ ДОСПРОСИТЬ

- 01 Что считать side-effect?
- 02 Как проверить?

ПОЛНАЯ СТАТЬЯ

<https://webpack.js.org/guides/tree-shaking/#mark-the-file-as-side-effect-free>

DYNAMIC IMPORT**CHUNK****CHUNK SPLITTING****BUNDLE SIZE**

ЧТО УЧИМ В ЭТОТ ДЕНЬ

DYNAMIC IMPORT

CHUNK

CHUNK SPLITTING

BUNDLE SIZE

ВОПРОСЫ ДНЯ

Q13 Что такое code splitting?

Q14 Что такое dynamic import?

Q15 Что такое chunk?

Q16 Как контролировать chunk splitting?

Q17 Как анализировать bundle size?

ВОПРОС Q13

Что такое code splitting?

ОПИСАНИЕ

Разбиение bundle на отдельные chunks, загружаемые по требованию. Уменьшает initial bundle. Реализация: `dynamic import('./module')`, `React.lazy`. Bundlers сами создают chunks.

КАК РАБОТАЕТ

- 01 `Dynamic import()`.
- 02 `React.lazy` + `Suspense`.
- 03 Route-based / component-level.
- 04 Manual chunks для vendor.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 SPA с большим initial bundle.
- 02 Lazy-loaded routes.
- 03 Heavy components.

ПОДВОДНЫЕ КАМНИ

- 01 Слишком мелкие chunks – много requests.
- 02 Без preload – задержка на nav.

МОГУТ ДОСПРОСИТЬ

- 01 Чем `dynamic import` от `React.lazy` отличается?
- 02 Что такое prefetch?

ПОЛНАЯ СТАТЬЯ

<https://web.dev/articles/code-splitting-suspense>

Что такое `dynamic import`?

ОПИСАНИЕ

`import('./module')` – async загрузка модуля, returns Promise. Bundler создаёт отдельный chunk. Браузер загружает on-demand. Основа code-splitting.

КАК РАБОТАЕТ

- 01 `import('./mod')` → Promise.
- 02 Async load.
- 03 Bundler creates chunk.
- 04 Native ESM feature.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Lazy components.
- 02 Conditional loading.
- 03 Plugin systems.

ПОДВОДНЫЕ КАМНИ

- 01 Dynamic путь без literal – bundler не сможет создать chunk.
- 02 No fallback for chunk-load error.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое import-maps?
- 02 Как обработать chunk-load error?

ПОЛНАЯ СТАТЬЯ

<https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/import>

ВОПРОС Q15

Что такое chunk?

ОПИСАНИЕ

Отдельный файл-fragment bundle, загружаемый отдельно. Bundler создаёт chunks для: dynamic imports, vendor (node_modules), shared (common между entry points).

КАК РАБОТАЕТ

- 01 Separate output file.
- 02 Dynamic / vendor / common types.
- 03 Loaded on-demand.
- 04 Hash в имени для cache-busting.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Code-splitting.
- 02 Long-term caching.

ПОДВОДНЫЕ КАМНИ

- 01 Слишком много chunks → overhead requests.
- 02 Слишком крупный — медленный initial.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое splitChunks?
- 02 Чем vendor chunk полезен?

ПОЛНАЯ СТАТЬЯ

<https://webpack.js.org/configuration/optimization/#optimizationsplitchunks>

Как контролировать chunk splitting?

ОПИСАНИЕ

Webpack: `optimization.splitChunks` с custom правилами (`cacheGroups`). Vite/Rollup: `manualChunks` function. Стратегия: vendor отдельно, common shared, route-based per-route.

КАК РАБОТАЕТ

- 01 `splitChunks.cacheGroups` (Webpack).
- 02 `manualChunks` (Vite/Rollup).
- 03 Vendor separation.
- 04 Route-based splitting.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Long-term caching.
- 02 Reduce duplicate code.

ПОДВОДНЫЕ КАМНИ

- 01 Слишком гранулярно → много мелких files.
- 02 Vendor-chunk обновляется при любом dep-update.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое `cacheGroups`?
- 02 Чем `splitChunks` автоматический?

ПОЛНАЯ СТАТЬЯ

<https://vitejs.dev/config/build-options.html#build-rollupoptions>

ВОПРОС Q17

Как анализировать bundle size?

ОПИСАНИЕ

Tools: webpack-bundle-analyzer, rollup-plugin-visualizer, source-map-explorer (no source-maps). Показывают визуальную карту bundle. CI-чек через size-limit / bundlwatch.

КАК РАБОТАЕТ

- 01 webpack-bundle-analyzer.
- 02 rollup-plugin-visualizer.
- 03 source-map-explorer.
- 04 size-limit в CI.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Bundle audit.
- 02 После добавления deps.
- 03 Регулярные проверки.

ПОДВОДНЫЕ КАМНИ

- 01 Без source-maps анализ неточный.
- 02 Tree-shake не работает для CommonJS.

МОГУТ ДОСПРОСИТЬ

- 01 Что показывает bundle-analyzer?
- 02 Чем bundlephobia полезен?

ПОЛНАЯ СТАТЬЯ

<https://github.com/webpack-contrib/webpack-bundle-analyzer>

SOURCE MAPS

CACHE BUSTING

LONG-TERM CACHING

ЧТО УЧИМ В ЭТОТ ДЕНЬ

SOURCE MAPS

CACHE BUSTING

LONG-TERM CACHING

ВОПРОСЫ ДНЯ

Q18 Что такое source map?

Q19 Какие риски у source maps в production?

Q20 Как работает cache busting?

Q21 Как именовать assets для long-term caching?

ВОПРОС Q18

Что такое source map?

ОПИСАНИЕ

Файл, mapping минифицированного кода обратно к оригинальному (TS, не-минифицированному JS). Browser DevTools использует для debug. Sentry – для красивых stack traces.

КАК РАБОТАЕТ

- 01 .map файл рядом с .js.
- 02 Mapping minified → original.
- 03 DevTools auto-loads.
- 04 Sentry/Bugsnag use.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Production debug.
- 02 Error tracking.

ПОДВОДНЫЕ КАМНИ

- 01 Public sourcemaps в prod = leak source.
- 02 Без sourcemaps stack traces бесполезны.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое source map types?
- 02 Где безопасно публиковать?

ПОЛНАЯ СТАТЬЯ

<https://web.dev/articles/source-maps>

Какие риски у source maps в production?

ОПИСАНИЕ

Public sourcemaps = публичный исходный код. Решение: hidden source maps (uploaded to Sentry, не serve публично), или нет sourcemaps в public, только в Sentry/CDN-private.

КАК РАБОТАЕТ

- 01 Public maps = source leak.
- 02 Sentry hidden sourcemaps.
- 03 Restrict via headers / IP.
- 04 No sourcemaps for sensitive code.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Production deployment.
- 02 Compliance audit.

ПОДВОДНЫЕ КАМНИ

- 01 Sourcemap published – источник видим.
- 02 No sourcemaps → can't debug prod.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое hidden source maps?
- 02 Чем sentry-cli upload-sourcemaps полезен?

ПОЛНАЯ СТАТЬЯ

<https://docs.sentry.io/platforms/javascript/sourcemaps/>

ВОПРОС Q20

Как работает cache busting?

ОПИСАНИЕ

Hash в имени файла (app.[hash].js). При изменении содержимого hash меняется → новый URL → bypassing browser cache. HTML ссылается на актуальные имена через manifest.

КАК РАБОТАЕТ

- 01 Hash в filename.
- 02 Content-hash → URL changes.
- 03 HTML manifest для resolve.
- 04 Long Cache-Control headers.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Любой production-deploy.
- 02 CDN-served assets.

ПОДВОДНЫЕ КАМНИ

- 01 Без hash → кэш браузера показывает старый.
- 02 Без manifest – HTML не обновляется.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое contenthash?
- 02 Чем chunkhash отличается?

ПОЛНАЯ СТАТЬЯ

<https://webpack.js.org/guides/caching/>

Как именовать assets для long-term caching?

ОПИСАНИЕ

filename: '[name].[contenthash].js' для JS, '[name].[contenthash][ext]' для assets.
Cache-Control: public, max-age=31536000, immutable. Vendor отдельно – реже меняется.

КАК РАБОТАЕТ

- 01 [name].[contenthash].ext.
- 02 Cache-Control: max-age=31536000, immutable.
- 03 Vendor отдельный chunk.
- 04 HTML – no-cache.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Production builds.
- 02 CDN-deploy.

ПОДВОДНЫЕ КАМНИ

- 01 HTML с long-cache → пользователь не видит обновлений.
- 02 Vendor + app в одном chunk → cache-bust на каждом релизе.

МОГУТ ДОСПРОСИТЬ

- 01 Чем immutable полезен?
- 02 Что такое CDN cache-headers?

ПОЛНАЯ СТАТЬЯ

<https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Cache-Control>

ДЕНЬ 6

НАГРУЗКА

СРЕДНИЙ

5 ВОПРОСОВ

DEPS**POLYFILLS****LEGACY****ENV**

ЧТО УЧИМ В ЭТОТ ДЕНЬ

DEPS

POLYFILLS

LEGACY

ENV

ВОПРОСЫ ДНЯ

Q22 Как оптимизировать dependencies?

Q23 Как работать с polyfills?

Q24 Как поддерживать legacy browsers?

Q25 Как организовать environment variables?

Q26 Какие env-переменные нельзя попадать в frontend bundle?

Как оптимизировать dependencies?

ОПИСАНИЕ

Tree-shake (esm-aware), per-method imports (lodash → lodash/debounce), замена тяжёлых (moment → date-fns/dayjs), audit unused (depcheck), bundlephobia.com перед добавлением.

КАК РАБОТАЕТ

- 01 Per-method imports.
- 02 Replace heavy deps.
- 03 Audit through bundle-analyzer.
- 04 bundlephobia check.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 After adding new lib.
- 02 Bundle-bloat audit.

ПОДВОДНЫЕ КАМНИ

- 01 `import _ from 'lodash'` тащит всё.
- 02 Дубли версий в monorepo.

МОГУТ ДОСПРОСИТЬ

- 01 Чем lodash-es от lodash отличается?
- 02 Что такое selective imports?

ПОЛНАЯ СТАТЬЯ

<https://bundlephobia.com/>

ВОПРОС Q23

Как работать с polyfills?

ОПИСАНИЕ

Стратегии: core-js + babel-preset-env (auto), browserslist для targeting, modern bundle для современных + legacy fallback (differential serving), conditional polyfills (polyfill.io).

КАК РАБОТАЕТ

- 01 core-js + babel-preset-env.
- 02 browserslist config.
- 03 Modern + legacy bundles.
- 04 polyfill.io conditional.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Browser compat.
- 02 Multi-browser SaaS.

ПОДВОДНЫЕ КАМНИ

- 01 Polyfills для всего → +KB.
- 02 Без targeting – лишние polyfills.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое browserslist?
- 02 Чем differential serving полезен?

ПОЛНАЯ СТАТЬЯ

<https://github.com/browserslist/browserslist>

Как поддерживать legacy browsers?

ОПИСАНИЕ

Modern bundle для современных + legacy fallback (через @vitejs/plugin-legacy / es5-adapter). Browserslist в config. Polyfills через core-js. Не bloat current users.

КАК РАБОТАЕТ

- 01 @vitejs/plugin-legacy.
- 02 Differential serving.
- 03 browserslist config.
- 04 Polyfills через core-js.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Public-facing apps.
- 02 Enterprise (IE11).

ПОДВОДНЫЕ КАМНИ

- 01 Modern + legacy = bigger total bundle.
- 02 IE11 поддержка тяжёлая.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое modulepreload?
- 02 Когда дропать IE?

ПОЛНАЯ СТАТЬЯ

<https://github.com/vitejs/vite/tree/main/packages/plugin-legacy>

ВОПРОС Q25

Как организовать environment variables?

ОПИСАНИЕ

Через .env файлы (.env / .env.development / .env.production). Только переменные с префиксом (VITE_ / REACT_APP_) попадают в bundle. Secrets – только server-side, не в bundle.

КАК РАБОТАЕТ

- 01 .env / .env.production.
- 02 Prefix VITE_ / NEXT_PUBLIC_.
- 03 Secrets – server-side only.
- 04 dotenv loader auto.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Multi-env deploys.
- 02 API URLs / public keys.

ПОДВОДНЫЕ КАМНИ

- 01 Server-secret в bundle = leak.
- 02 Без префикса – не пробрасывается.

МОГУТ ДОСПРОСИТЬ

- 01 Чем NEXT_PUBLIC_ важен?
- 02 Что такое env-vault?

ПОЛНАЯ СТАТЬЯ

<https://vitejs.dev/guide/env-and-mode.html>

Какие env-переменные нельзя попадать в frontend bundle?

ОПИСАНИЕ

Secrets: API-keys (private), DB-credentials, JWT-secret, AWS-keys, encryption-keys.
Frontend bundle = публичный код – всё доступно через DevTools.

КАК РАБОТАЕТ

- 01 DB-passwords, API-secrets.
- 02 JWT signing keys.
- 03 AWS / cloud credentials.
- 04 Internal endpoints.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Code review.
- 02 Security audit.

ПОДВОДНЫЕ КАМНИ

- 01 Stripe-secret-key в bundle = breach.
- 02 JWT-secret в bundle = forge tokens.

МОГУТ ДОСПРОСИТЬ

- 01 Где хранить secrets?
- 02 Что такое edge-config?

ПОЛНАЯ СТАТЬЯ

<https://owasp.org/www-project-top-ten/>

ДЕНЬ 7

НАГРУЗКА

ТЯЖЁЛЫЙ

4 ВОПРОСА

BUILD ENVS**WHITE-LABEL****SHARED PKG****DEDUP**

ЧТО УЧИМ В ЭТОТ ДЕНЬ

BUILD ENVS

WHITE-LABEL

SHARED PKG

DEDUP

ВОПРОСЫ ДНЯ

Q27 Как настроить build для нескольких environments?

Q28 Как проектировать build для white-label UI?

Q29 Как проектировать shared UI package?

Q30 Как избежать dependency duplication?

Как настроить build для нескольких environments?

ОПИСАНИЕ

Mode-flag (--mode=staging) → loads .env.staging. Build-once-deploy-many: один artifact, env через runtime-config (manifest.json или window.ENV). Vercel / Netlify env-vars per env.

КАК РАБОТАЕТ

- 01 --mode flag.
- 02 Build-once-deploy-many через runtime-config.
- 03 Vercel ENV per branch.
- 04 Environment-specific dashboards.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Multi-env CI/CD.
- 02 Staging + prod.

ПОДВОДНЫЕ КАМНИ

- 01 Per-env build → different artifacts → harder to rollback.
- 02 Hardcoded URLs.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое runtime-config?
- 02 Чем build-once полезен?

ПОЛНАЯ СТАТЬЯ

<https://vitejs.dev/guide/env-and-mode.html#modes>

ВОПРОС Q 28

Как проектировать build для white-label UI?

ОПИСАНИЕ

Theme tokens (CSS-vars) per brand. Сборка с brand-config (--brand=acme). Shared core, white-label-overrides. Иногда multi-tenant – один build, brand определяется runtime.

КАК РАБОТАЕТ

- 01 Theme-tokens per brand.
- 02 Brand-config flag.
- 03 Multi-tenant runtime-config.
- 04 Asset-overrides per brand.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Multi-tenant SaaS.
- 02 Reseller products.

ПОДВОДНЫЕ КАМНИ

- 01 Brand-overrides ломают base-styles.
- 02 Без isolation – кросс-загрязнение.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое CSS-vars темы?
- 02 Чем multi-tenant runtime удобнее?

ПОЛНАЯ СТАТЬЯ

<https://web.dev/articles/color-scheme>

Как проектировать shared UI package?

ОПИСАНИЕ

Monorepo-package: ESM + types + tree-shake-friendly per-component exports. SemVer release. CSS как side-effect-free через CSS-modules / styled / vanilla-extract. Storybook с docs.

КАК РАБОТАЕТ

- 01 ESM + types.
- 02 Per-component exports.
- 03 CSS-modules / vanilla-extract.
- 04 SemVer + Changesets.
- 05 Storybook docs.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Internal DS.
- 02 Open-source UI lib.

ПОДВОДНЫЕ КАМНИ

- 01 Big monolithic export – нет tree-shake.
- 02 CSS-side-effects ломают tree-shake.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое vanilla-extract?
- 02 Чем Changesets полезен?

ПОЛНАЯ СТАТЬЯ

<https://vanilla-extract.style/>

ВОПРОС Q30

Как избежать dependency duplication?

ОПИСАНИЕ

Monorepo single-version policy (через npm workspaces, pnpm). hoist deps. Module Federation singleton. Audit через npm ls / pnpm why. Resolutions / overrides в package.json.

КАК РАБОТАЕТ

- 01 Monorepo single-version.
- 02 pnpm workspace + hoist.
- 03 MF singleton: { react: { singleton: true } }.
- 04 resolutions / overrides.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Monorepos.
- 02 Module Federation.
- 03 Multi-package projects.

ПОДВОДНЫЕ КАМНИ

- 01 Multiple React copies → broken hooks.
- 02 Без resolutions – silent duplicates.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое pnpm hoist?
- 02 Чем resolutions полезен?

ПОЛНАЯ СТАТЬЯ

<https://pnpm.io/feature-comparison>

СДАЛ ЛИ ЗАЧЁТ

ПРОЙДИ ВСЛУХ БЕЗ ПОДГЛЯДЫВАНИЙ ЕСЛИ ЗАСТРЯЛ – ВЕРНИСЬ В НУЖНЫЙ БИЛЕТ

- ☐ [] Описать bundler / Vite vs Webpack / dev-server / ESM / HMR (Q1-Q6)
- ☐ [] Описать prod build / Rollup / esbuild (Q7, Q8, Q9)
- ☐ [] Описать tree shaking / side effects / code splitting (Q10, Q11, Q12)
- ☐ [] Описать dynamic import / chunk / chunk splitting / bundle analysis (Q13-Q17)
- ☐ [] Описать source maps / cache busting (Q18-Q21)
- ☐ [] Оптимизировать deps / polyfills / legacy / env-vars (Q22-Q26)
- ☐ [] Описать build envs / white-label / shared package / dedup (Q27-Q30)